Frogger

CS Project Documentation

This document explains my Frogger game: what it does, how to play it, and how the code is put together. The whole game is one Python file using pygame.

1. Introduction

For this project I built a Frogger-style arcade game in Python. The player controls a frog that has to cross busy roads and a river full of moving logs to reach lily pads at the top of the screen. It is inspired by the classic 1981 Konami arcade game, but I wrote everything from scratch with simple shapes instead of sprites.

I chose this because it hits a good balance for a class project: the rules are easy to understand, but the movement on logs and collision with cars still makes you think about timing and positioning.

2. What you need to run it

- Python 3.9 or newer
- pygame library (install with: pip install pygame)
- A computer with a screen (desktop window game, not a web game)

From the project folder, run:

```
python3 frogger.py
```

3. How to play

The frog starts at the bottom on a safe green strip. The goal is to hop upward through different lanes and land on every lily pad at the top. You have 3 lives. Losing all lives ends the game.

Lane types from bottom to top:

- Safe zones (green): no danger, catch your breath
- Road (gray): cars drive left and right; touching one kills you
- River (blue): you must stand on logs; open water drowns you
- Goals (top green): lily pads; land on each one to clear the level

Movement is grid-based. Each key press moves the frog exactly one tile. There is a short hop cooldown so you cannot move too fast.

4. Controls

- Arrow keys or WASD: hop one tile
- Space or Enter: start game / continue after dying
- Close the window to quit

5. Scoring and levels

You earn points every time you reach a lily pad (100 plus a small bonus based on how high up you are). When all lily pads on a level are filled, you advance to the next level. Cars and logs speed up a little each level, so it gets harder over time.

6. Log mechanics (important part)

The trickiest part of Frogger is the river. I modeled it after the original game: each log has a fixed number of slots (1, 2, or 3) that are exactly one tile wide. You can only stand on those slots, not in the gap between logs or in the middle of a log where there is no slot.

When you hop onto a river row, the game checks if your column lines up with a slot on any log in that row. If yes, the frog locks to that slot and rides the log as it moves. Your screen position is calculated from the log position plus the slot offset, so you move smoothly with the log instead of sliding around on your own.

You die in the river if you hop into water with no log under you, or if the log carries your slot off the edge of the screen.

7. Code structure

The game is in frogger.py as a single file (~480 lines). I used camelCase for variable names and left comments explaining what each section does.

Main classes:

- Frog: player state, hop logic, riding logs, collision rect
- Car: position, speed, width; wraps around when leaving the screen
- Log: slot count, movement, slot position math
- Lanes: creates all cars and logs, tracks lily pads, attaches frog to logs

8. Game loop

The main() function runs a standard pygame loop at 60 FPS:

- 1. Read keyboard events (hops, start, restart)
- 2. Update car and log positions
- 3. Update frog (check drowning, car hits)
- 4. Draw background lanes, logs, cars, frog, HUD, and overlay text
- 5. Flip the display and repeat

Game state is tracked with a string: title, play, dead, win, or gameover. That controls what input is accepted and what gets drawn on screen.

9. Drawing

There are no PNG or JPG files. Everything is drawn with pygame primitives: rectangles for roads, cars, and logs; ellipses for the frog and lily pads; circles for headlights and eyes. The HUD bar at the top shows score, lives, and level so it does not cover the goal row.

10. Project files

- frogger.py: the entire Frogger game
- README.md: quick start guide
- FROGGER_PROJECT.pdf: this document
- requirements.txt: Python dependencies

Note: this repository also contains jun_world/, a larger story game from an earlier iteration of the project. That is separate from Frogger and is not required to run this game.

11. What I learned

Building this taught me how to structure a game loop, handle collision detection with pygame.Rect, and attach a player to a moving platform without weird glitches. The log slot system was the hardest part. Free-floating pixel movement kept breaking until I snapped the frog to discrete slots on each log, like the real arcade game.

12. Possible improvements

- Add sound effects and music
- High score saved to a file
- Turtles that dive underwater after a few seconds
- A proper start menu with difficulty options
- Sprite images instead of drawn shapes

End of document